PROGRAMACIÓN DE ENTORNOS MULTIMEDIA

Semana 6

Físicas Avanzadas, Mecánicas de Control y Cámara Dinámica

Mg. Gabriel Guerrero Arellano

Universidad cooperativa de Colombia

Sesión teórica y práctica de 3 horas.

Gestión de entornos físicos complejos, controladores, cámara en 3D e instanciación dinámica (disparo de proyectiles).



¿Cómo trabajaremos hoy?

01 Conexión con la ruta

Ruta de aprendizaje y clases anteriores.

02 Marco teórico profundizado

Físicas avanzadas, impulsos y cámaras.

03 Herramientas de desarrollo

Preparación del entorno y tecnologías de trabajo.

04 Laboratorio, parte 0

Preparación del entorno Vite y dependencias.

05 Laboratorio práctico

Escenario, personaje, controles, eventos y cámara.

06 ¡BONUS!

Instanciación y disparo de proyectiles.

07 Evidencia de aprendizaje y conclusiones

Validación del aprendizaje y cierre de la sesión.



¿Dónde estamos en nuestra ruta de aprendizaje?

Semana 5

En la Semana 5 estudiamos la introducción a simulaciones físicas, gravedad básica y detección de colisiones simples.

Hoy · Momento 2

Físicas, Control y Carga de Assets.

Nivel técnico más alto

Elevamos el nivel técnico: implementaremos mecánicas de salto por impulsos, escucha de eventos de colisión y una cámara seguidora en tercera persona.

Renderizado Gráfico vs. Simulación Física

Marco Teórico 1: Arquitectura de Motores y Sincronización

Cannon-es · simulación

La física (Cannon-es) corre en **pasos de tiempo fijos (Fixed Timestep)**, independientemente de la tasa de refresco del monitor.

Calcula fuerzas, momentos de inercia y colisiones entre cuerpos rígidos (Body).

Three.js · renderizado

Bucle asíncrono: Three.js renderiza según la **tasa de refresco del monitor**.

La escena visual proyecta geometrías (Mesh).

Body	Transferencia	Mesh
Cuerpo rígido calculado por la física.	Copia vectores de posición y rotaciones cuaterniónicas en cada fotograma.	Representación gráfica sincronizada con el cuerpo físico.

Optimización de Colisiones en Entornos Masivos

Marco Teórico 2: Algoritmos de Colisión y Optimización



Broadphase · fase amplia

Descarta rápidamente objetos lejanos mediante volúmenes envolventes (AABB), con métodos como Sweep and Prune (SAP).

Narrowphase · fase estrecha

Calcula con precisión puntos de contacto e intersección geométrica solo entre entidades cercanas.

Estado «Sleep»

Congela cálculos numéricos en cuerpos cuya velocidad cae bajo un umbral y libera recursos de CPU.

Dinámica de Movimiento y Escucha de Impactos

Marco Teórico 3: Impulsos, Fuerzas y Eventos

Movimiento	Velocidad directa	Impulsos
Operación	Modificar la velocidad lineal.	`applyImpulse`
Respuesta	Control responsivo (arcade).	Física realista con inercia.



Mecánica de salto

Aplicación de una fuerza instantánea en el **eje Y**, corregida por el vector de gravedad.

$$g = -9.81 \text{ m/s}^2$$

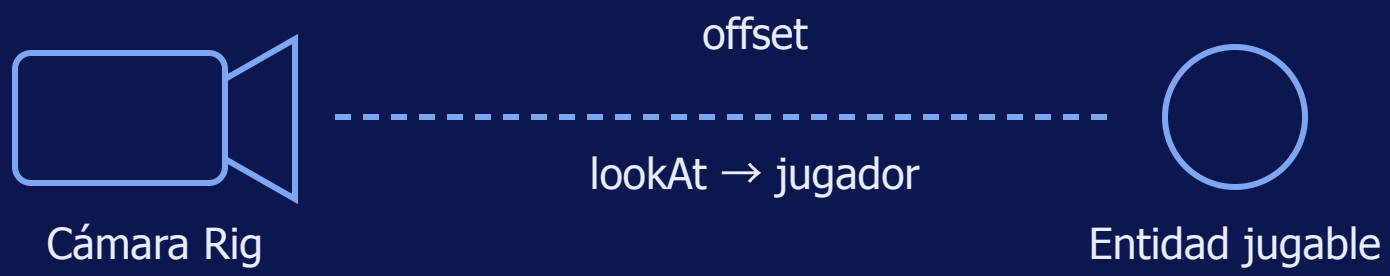


Listeners de colisión

Los motores emiten eventos (`collide`) para activar respuestas sonoras, cambios de material o contadores de puntuación.

Arquitectura de Cámara en Tercera Persona

Marco Teórico 4: Sistema de Cámara Seguidora



△ Cámara Rig

Mantiene un vector de desfasamiento (**offset**) respecto a la entidad jugable.

≈ Interpolación Lerp

Suaviza la trayectoria de la cámara para evitar saltos bruscos.

◎ Orientación dinámica

`lookAt` apunta a las coordenadas en tiempo real del jugador y mantiene el enfoque centrado.

Nuestro Arsenal Tecnológico

Node.js y Vite

Entorno de ejecución y empaquetado rápido.

Three.js

Motor gráfico para renderizado 3D y luces.

Cannon-es

Motor de física 3D: gravedad e impulsos.

JavaScript ES6

Captura de eventos del DOM y lógica modular.



Laboratorio 0.1: Preparación del Entorno Vite

Archivo destino: Terminal de VS Code · carpeta del proyecto.

Ejecuta los comandos en orden para crear e instalar el entorno.

```
# 1. Crear el proyecto Vite en la carpeta actual
npm create vite@latest . -- --template vanilla
# 2. Instalar las dependencias base de Node
npm install
# 3. Instalar nuestros motores 3D y de físicas
npm install three cannon-es
```

Laboratorio 0.2: Limpieza de Plantilla y Archivos

Archivo destino: `index.html` (raíz del proyecto).

Reemplaza `index.html` para dejar el Canvas 3D a pantalla completa.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Laboratorio Físicas Avanzadas</title>
  </head>
  <body style="margin: 0; overflow: hidden;">
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

Laboratorio 1: Mundo y Pila de Cajas (Parte 1/2)

Archivo destino: `src/main.js` (Borra todo el código previo).

Escena, cámara, luces y renderizado base.

```
import * as THREE from 'three';
import * as CANNON from 'cannon-es';
// 1. CONFIGURACIÓN BASE Y LUCES
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x87ceeb);
const camera = new THREE.PerspectiveCamera(75, window.innerWidth/window.innerHeight, 0.1, 1000);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);
scene.add(new THREE.AmbientLight(0xffffff, 0.6));
const dirLight = new THREE.DirectionalLight(0xffffff, 0.8);
dirLight.position.set(10, 20, 10);
scene.add(dirLight);
```

Laboratorio 1: Mundo y Pila de Cajas (Parte 2/2)

Archivo destino: `src/main.js` (continuación).

Suelo físico y generación de la pila de cajas.

```
// 2. MUNDO FÍSICO Y SUELO
const world = new CANNON.World({ gravity: new CANNON.Vec3(0, -9.81, 0) });
const groundBody = new CANNON.Body({ mass: 0, shape: new CANNON.Plane() });
groundBody.quaternion.setFromEuler(-Math.PI / 2, 0, 0);
world.addBody(groundBody);
const groundMesh = new THREE.Mesh(
    new THREE.PlaneGeometry(60, 60),
    new THREE.MeshStandardMaterial({ color: 0x333333 })
);
groundMesh.rotation.x = -Math.PI / 2;
scene.add(groundMesh);
// 3. CAJAS DINÁMICAS
const objects = [];
for (let i = 0; i < 20; i++) {
    const body = new CANNON.Body({ mass: 1, shape: new CANNON.Box(new CANNON.Vec3(0.5, 0.5, 0.5)) });
    body.position.set((Math.random() - 0.5) * 6, 2 + i * 1.2, (Math.random() - 0.5) * 6);
    world.addBody(body);
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(1, 1, 1), new THREE.MeshStandardMaterial({ color: 0x00ff88 }));
    scene.add(mesh);
    objects.push({ body, mesh });
}
```

Laboratorio 2: Configuración del Personaje Principal

Archivo destino: `src/main.js` (continuación).

Entidad esférica del jugador y ajuste de fricción.

```
// 4. ENTIDAD JUGADOR
const playerRadius = 0.8;
const playerShape = new CANNON.Sphere(playerRadius);
const playerBody = new CANNON.Body({
    mass: 5,
    shape: playerShape,
    linearDamping: 0.3
});
playerBody.position.set(0, 2, 8);
world.addBody(playerBody);
const playerMesh = new THREE.Mesh(
    new THREE.SphereGeometry(playerRadius, 32, 32),
    new THREE.MeshStandardMaterial({ color: 0xff0055, roughness: 0.3 })
);
scene.add(playerMesh);
```

Laboratorio 3: Control Multitecla y Salto

Archivo destino: `src/main.js` (continuación).

Controles multitecla y salto por impulso.

```
// 5. EVENTOS DE TECLADO
const keys = { w: false, a: false, s: false, d: false, space: false };
let canJump = false;
window.addEventListener('keydown', (e) => {
    if (e.key.toLowerCase() === 'w') keys.w = true;
    if (e.key.toLowerCase() === 'a') keys.a = true;
    if (e.key.toLowerCase() === 's') keys.s = true;
    if (e.key.toLowerCase() === 'd') keys.d = true;
    if (e.code === 'Space' && canJump) {
        playerBody.applyImpulse(new CANNON.Vec3(0, 25, 0), playerBody.position);
        canJump = false;
    }
});
window.addEventListener('keyup', (e) => {
    if (e.key.toLowerCase() === 'w') keys.w = false;
    if (e.key.toLowerCase() === 'a') keys.a = false;
    if (e.key.toLowerCase() === 's') keys.s = false;
    if (e.key.toLowerCase() === 'd') keys.d = false;
});
```

Laboratorio 4: Detección de Colisiones en Tiempo Real

Archivo destino: src/main.js (continuación).

Detección de contacto y respuesta visual al impacto.

```
// 6. COLISIONES EN TIEMPO REAL
playerBody.addEventListener('collide', (event) => {
    const contact = event.contact;
    if (contact.bi.id === playerBody.id && contact.ni.y > 0.5) {
        canJump = true;
    } else if (contact.bj.id === playerBody.id && contact.ni.y < -0.5) {
        canJump = true;
    }
    if (event.contact.getImpactVelocityAlongNormal() > 3) {
        playerMesh.material.color.setHex(0xffff00);
        setTimeout(() => playerMesh.material.color.setHex(0xff0055), 150);
    }
});
```

ejecutar en la terminal de vscode:

1. npm run dev
2. abrir en el navegador la url: http://localhost:5173/
3. se mueve con las teclas : W,A,S,D
4. probamos colisiones realistas.



Laboratorio Bonus: Disparo interactivo (Parte 1/2)

Archivo destino: `src/main.js` (antes del bucle `animate`).

Dirección, física y velocidad inicial del proyectil.

```
// BONUS: MECÁNICA DE DISPARO (Instanciación dinámica)
window.addEventListener('click', () => {
    // 1. Obtener dirección hacia donde mira la cámara
    const shootDirection = new THREE.Vector3();
    camera.getWorldDirection(shootDirection);
    const projRadius = 0.3;
    // 2. Crear física del proyectil
    const projBody = new CANNON.Body({ mass: 2, shape: new CANNON.Sphere(projRadius) });
    projBody.position.copy(playerBody.position);
    projBody.position.y += 1; // Disparar un poco por encima del jugador
    // 3. Aplicar velocidad inicial agresiva
    const shootVelocity = 35;
    projBody.velocity.set(
        shootDirection.x * shootVelocity,
        shootDirection.y * shootVelocity,
        shootDirection.z * shootVelocity
    );
    world.addBody(projBody);
```

Laboratorio Bonus: Disparo interactivo (Parte 2/2)

Archivo destino: `src/main.js` (continuación, antes de `animate`).

Visual del proyectil y sincronización con el bucle.

```
    // 4. Crear visual del proyectil (Naranja)
    const projMesh = new THREE.Mesh(
        new THREE.SphereGeometry(projRadius, 16, 16),
        new THREE.MeshStandardMaterial({ color: 0xffa500 })
    );
    scene.add(projMesh);
    // 5. ¡Añadir al arreglo existente para que se sincronice mágicamente en el animate!
    objects.push({ body: projBody, mesh: projMesh });
});
```

Laboratorio 5: Animación y Cámara (Parte 1/2)

Archivo destino: `src/main.js` (función final del archivo).

Controles, físicas y sincronización de objetos.

```
// 7. BUCLE DE ANIMACIÓN
function animate() {
    requestAnimationFrame(animate);
    world.fixedStep();
    const speed = 8;
    if (keys.w) playerBody.velocity.z = -speed;
    if (keys.s) playerBody.velocity.z = speed;
    if (keys.a) playerBody.velocity.x = -speed;
    if (keys.d) playerBody.velocity.x = speed;
    if (!keys.w && !keys.s) playerBody.velocity.z *= 0.9;
    if (!keys.a && !keys.d) playerBody.velocity.x *= 0.9;
    playerMesh.position.copy(playerBody.position);
    playerMesh.quaternion.copy(playerBody.quaternion);
    groundMesh.position.copy(groundBody.position);
    groundMesh.quaternion.copy(groundBody.quaternion);
    // Aquí se animan automáticamente tanto las cajas como los proyectiles
    objects.forEach(obj => {
        obj.mesh.position.copy(obj.body.position);
        obj.mesh.quaternion.copy(obj.body.quaternion);
    });
```

Laboratorio 5: Animación y Cámara (Parte 2/2)

Archivo destino: `src/main.js` (continuación).

Cámara en tercera persona y render final.

```
    const cameraOffset = new THREE.Vector3(0, 6, 12);
    camera.position.copy(playerBody.position).add(cameraOffset);
    camera.lookAt(playerMesh.position);
    renderer.render(scene, camera);
}
animate();
```



Ejecución y Validación Técnica del Laboratorio

Evidencia de aprendizaje

✓ Ejecución

Escribe npm run dev en la terminal, presiona Enter y abre el enlace local.

✓ Validación mecánica

El jugador se desplaza con WASD, salta con Espacio y reacciona a colisiones.

✓ Validación del BONUS

Haz clic en el lienzo (canvas). Debe instanciarse una esfera naranja que saldrá disparada a gran velocidad destruyendo la formación de cajas.

✓ Sincronización

Al estar dentro del mismo bucle (arreglo objects), nota cómo los proyectiles interactúan matemáticamente a la perfección con el resto del mundo.



Cierre de Sesión y Proyección a la Semana 7

Logramos un **hito crucial**: integrar el ecosistema Vite para crear un prototipo jugable interactivo con físicas, control, cámara y generación dinámica de elementos.

Comprobamos que al **separar la lógica física (Cannon) de la gráfica (Three.js) mediante un arreglo centralizado**, añadir nuevas mecánicas (como el disparo) resulta sorprendentemente sencillo.

Conexión con la Semana 7

En la siguiente clase avanzaremos a la **Importación de Assets (.gltf / .glb)**, sustituyendo nuestras primitivas por personajes y mundos modelados profesionalmente.